

CareerPilot

Stack & Justification Report

Executive Summary: CareerPilot is a full-stack AI career co-pilot built around one core principle: user career guidance should be grounded in the user's own CV, not generated in a vacuum. The stack is appropriate because it separates presentation, business logic, persistent state, caching, and retrieval-backed AI in a way that is easy to scale and easy to explain.

1. Product Goal

CareerPilot helps job seekers with five connected workflows:

- 1. Upload and analyze a CV
- 2. Search jobs and score them against the CV
- 3. Chat with an AI career assistant
- 4. Track applications in a Kanban-style pipeline
- 5. Generate polished PDF or DOCX CVs from structured form input

This is not a single AI demo feature. It is a career workflow system, and the architecture reflects that.

2. Stack Overview

Layer	Technology	Justification
Frontend	Next.js 16 + React 19 + TypeScript	Interactive, route-driven, stateful UI with strong typing for API contracts.
Backend	FastAPI + Uvicorn	Typed request/response models, async-friendly, auto API docs for feature validation.
Database	PostgreSQL + SQLAlchemy	Relational data with foreign keys, transactional writes, and queryable history.
Caching	Redis	In-memory cache for job results, fit scores, and session tokens — reduces latency and repeated computation.

Layer	Technology	Justification
Vector Store	ChromaDB	Per-user CV retrieval with collection isolation — no cross-user leakage.
Embeddings	sentence-transformers (all-MiniLM-L6-v2)	Local, free, portable — avoids paying for embeddings.
AI Orchestration	LangChain + Google Gemini	Tool calling, memory, routing, and recovery from model edge cases.
Document Output	ReportLab + python-docx	Polished PDF output and editable DOCX export without a separate pipeline.
Infrastructure	Docker Compose	Four-service orchestration (DB, Redis, backend, frontend) with volume caching.

3. Why This Stack Fits the Problem

3.1 Next.js Frontend

The app is highly interactive, route-driven, and stateful — it needs protected pages, a persistent shell, a dashboard, a chat experience, drag-and-drop tracking, and multi-step forms. Next.js App Router is a strong match because each feature lives in its own route while sharing the same shell and context providers. TypeScript is especially valuable because the frontend consumes many typed API responses: auth, chat history, CV status, job cards, applications, todos, and dashboard stats.

3.2 FastAPI Backend

The product exposes many JSON endpoints, file uploads, and authenticated operations. The backend also needs async-friendly integrations with LLM calls, vector search, and external job APIs. FastAPI's typed request/response models make the contract clear. The backend is split into routers and services — routers handle API boundaries; services handle business logic such as parsing CVs, computing fit score, searching jobs, generating documents, and managing memory.

3.3 PostgreSQL for Persistence

The application has real relational data: users, profiles, chat messages, applications, todos, and activities. These entities are naturally user-scoped and benefit from foreign keys, transactional writes, and queryable history. Conversations must survive restarts, and the dashboard needs aggregated stats across user activity. A relational database is a better fit than a purely in-memory or document-only store.

3.4 Redis for Caching

Redis is used as an in-memory caching layer between the API and the heavier backend services. Job search results are cached by query key so repeated searches for the same role or location do not trigger redundant external API calls. Fit scores are cached per user-job pair so the scoring formula does not re-execute on every page load. Session tokens and JWT allowlists are also managed through Redis, enabling fast token validation and immediate invalidation on logout. This reduces latency, protects rate-limited external services, and keeps the PostgreSQL load low.

3.5 ChromaDB for CV Retrieval

ChromaDB provides per-user CV retrieval without forcing a call to an external vector service. Each user gets their own collection, which supports isolation and prevents cross-user leakage — matching the product's privacy-first positioning.

3.6 LangChain for Agent Orchestration

The chat experience is not just open-ended generation. It needs tool calling, memory, routing, and recovery from model edge cases. Simple messages go through a fast direct LLM path; career-specific messages go through a tool-using agent. Empty agent outputs are recovered by synthesizing from intermediate tool steps. Conversation memory is persisted in PostgreSQL instead of kept only in RAM.

3.7 Local Sentence Embeddings

The use of all-MiniLM-L6-v2 through sentence-transformers avoids paying for embeddings, reduces dependency on external APIs, and makes CV ingestion cheaper and more portable — a high-value tradeoff for a hackathon or student project.

3.8 ReportLab and python-docx

The CV builder needs both a polished PDF output and an editable DOCX output. ReportLab is appropriate for high-control PDF layout, while python-docx covers Word export without needing a separate document pipeline.

4. Architecture Summary

4.1 Frontend Layer

- Dashboard with career stats and activity feed
- Profile page for CV upload and analysis
- Chat page for AI assistance
- Jobs page for job discovery and fit scoring
- Tracker page for application management (Kanban)
- CV Builder page for generating PDF and DOCX documents

4.2 Backend Layer

- Auth router — registration, login, and current-user retrieval
- CV router — upload, status, and clear operations
- Chat router — assistant interaction and history management
- Jobs router — search, ranking, and cached result retrieval
- Tracker router — applications, todos, and dashboard stats
- CV Builder router — PDF and DOCX generation

4.3 Data Layer Schema

Entity / Store	Purpose
users	Authentication
user_profiles	CV metadata
chat_messages	Conversational persistence
applications	Kanban tracker state
todos	Career tasks

Entity / Store	Purpose
activities	Dashboard activity feed
Redis: job_cache	Cached job search results keyed by query + location
Redis: score_cache	Cached fit scores keyed by user ID + job ID
Redis: token_store	JWT allowlist for fast validation and logout invalidation

5. Key Design Decisions

Deterministic Fit Scoring

The job fit score is computed with a formula using skill, experience, and education signals. A reviewer can inspect the score breakdown and understand why a role ranked high or low. This is better than asking the model to invent a score, because the user can trust the result.

Section-Aware CV Parsing

The CV processor detects resume sections, chunks the content, and preserves metadata. This improves retrieval quality because the assistant can cite more relevant fragments and the profile page can report detected sections back to the user.

Two-Tier Chat Routing

Greetings and small talk avoid unnecessary agent overhead. Career questions trigger retrieval, scoring, and search tools. This is the right balance for a responsive product.

Database-Backed Memory

Persisting chat history in PostgreSQL survives restarts, supports multi-user usage, and avoids the brittleness of pure in-memory state — making the assistant feel like a real product, not a one-off demo.

Redis Cache-Aside Pattern

Job results and fit scores are fetched from Redis first; on a miss the backend computes the result, writes it to Redis with a TTL, and returns it to the client. This keeps the system fast under repeated queries without serving stale data indefinitely. Token invalidation on logout is handled with an immediate Redis key deletion, making auth revocation instant without waiting for JWT expiry.

6. Risks & Tradeoffs

- The product depends on external services for some job search and LLM functionality — rate limits and missing API keys must be handled gracefully.
- The CV pipeline is only as good as the uploaded document quality — malformed or image-only resumes will be weaker inputs.
- Redis adds an infrastructure dependency. If Redis is unavailable the cache layer must degrade gracefully so the product still functions.
- The current architecture is optimized for a single product instance and small-scale use, not a large enterprise multi-tenant deployment.

- The custom frontend styling is expressive, but future consistency depends on maintaining the project's own design system.
-

7. Bottom Line

CareerPilot's stack is justified because each major technology is used where it has the highest leverage: Next.js and TypeScript for a structured, interactive career product; FastAPI and PostgreSQL for reliable API and data persistence; **Redis for fast caching of job results, fit scores, and session tokens;** ChromaDB and sentence-transformers for cheap, private retrieval; LangChain and Gemini for the assistant layer; ReportLab and python-docx for professional output generation. The overall design is not just modern — it is aligned with the actual behavior of the application.